Four Number Sum

AlgoExpert · Hard · Arrays

The problem

- All quadruplets of distinct integers summing to target (any order).
- Store pair sums seen before index i, combine with pairs after i, so each quadruplet is reported once.

Intuition

- Split the quadruplet into two pairs, and use a hash map of pair sums seen strictly before index i to avoid duplicates.

Brute force vs optimal

- Brute force: four loops → $O(n^4)$
- Sort + two pointers → $O(n^3)$
- Pair-sum hash map → $O(n^2)$ average

Approach

- 1. For each i, look at pairs i with j after it and check target minus their sum in the map
- 2. Then add pairs k with i, k before i, to the map
- 3. This ordering prevents reusing indices

Worked example




```
quads found: none
pair sums stored: [13]
```

i = 1: look ahead, then store pairs ending at 6

Worked example




```
quads found: [[7, 6, 4, -1]]
pair sums stored: [10, 11, 13]
```

i = 2: look ahead, then store pairs ending at 4

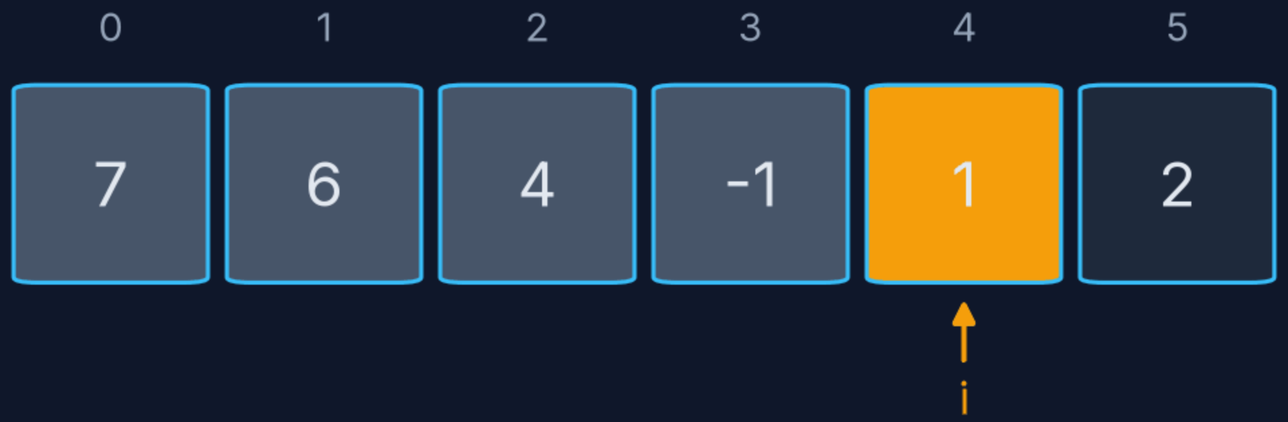
Worked example




```
quads found: none
pair sums stored: [3, 5, 6, 10, 11, 13]
```

i = 3: look ahead, then store pairs ending at -1

Worked example




```
quads found: [[7, 6, 1, 2]]
pair sums stored: [0, 3, 5, 6, 7, 8, 10, 11, 13]
```

i = 4: look ahead, then store pairs ending at 1

Java solution

```java
public class FourNumberSum {
    public static List<Integer[]> fourNumberSum(int[] a, int target) {
        Map<Integer, List<int[]>> pairs = new HashMap<>();
        List<Integer[]> out = new ArrayList<>();
        for (int i = 1; i < a.length - 1; i++) {
            for (int j = i + 1; j < a.length; j++) {
                List<int[]> ps = pairs.get(target - a[i] - a[j]);
                if (ps != null) for (int[] p : ps) out.add(new Integer[]{p[0], p[1], a[i], a[j]});
            }
            for (int k = 0; k < i; k++) pairs.computeIfAbsent(a[i] + a[k], x -> new ArrayList<>()).add(new int[]{a[k], a[i]});
        }
        return out;
    }
}
```

Complexity

- Time: O(n^2) avg
- Space: O(n^2)

Edge cases & pitfalls

- Only store pairs *before* i to avoid reusing elements
- Worst case $O(n^3)$ when many pairs share a sum
- Distinct quadruplets only

Interview tips & follow-ups

- Explain the look-ahead/store ordering
- Follow-up: k-sum generalisation

Recap

- Split the quadruplet into two pairs, and use a hash map of pair sums seen strictly before index i to avoid duplicates.
- Complexity: O(n^2) avg